Event Desk

Design Patterns Implementation Project Report

Repository: Ruslankyzy-Aldina/final_project_sdp Contribution branch: feature/gui-facade Baseline: a924bcb, feature/booking-storage Verification date: 8 November 2026

1. Introduction

1.1 Background

Event registration combines capacity rules, pricing, booking status changes and reporting. If a graphical interface implements those rules itself, every screen becomes coupled to storage and calculation details. Event Desk separates these responsibilities using a facade, exporter creation through Factory Method, and the pricing strategies already present in the repository.

1.2 Objectives

- Preserve the existing Booking model, statuses, pricing strategies and tests.

- Provide a graphical workflow for events, participants, bookings and summaries.

- Export the same summary to TXT, JSON and HTML through a common interface.

- Reject over-capacity bookings without partially changing application state.

- Supply demo data, repeatable tests and documented demonstration scenarios.

1.3 Scope

This contribution is a Java 25 desktop application using Swing and Maven. It implements Factory Method and Facade and integrates the existing Strategy pattern. It does not claim to implement the six patterns or document-conversion features described in the supplied formatting example. Team names and course group details are not invented in this report.

2. Project Overview

The inspected baseline contained Booking, BookingStatus, PriceCalculator, three pricing strategies, two JUnit tests and a placeholder Main. There were no event or participant models, storage services, GUI or report files. This branch adds those layers without changing the existing booking and pricing source files.

Users can create events, register participants, reserve seats, change booking status, view an immutable summary and save a report. Demo data contains three events and three participants. The conference has eight reserved seats out of twelve and a total of 35750.00 KZT. The workshop deliberately has no available seats so scenario 8 can be demonstrated immediately.

3. Design Patterns Implementation

3.1 Pattern table

Pattern	Role and classes	Contribution
Factory Method	ReportExporter; TxtExporter, JsonExporter, HtmlExporter; ExporterFactory and DefaultExporterFactory	New: centralizes exporter creation and allows the creator operation to use an overridden factory method
Facade	EventFacade coordinating EventService, BookingService, store and exporters	New: gives every GUI action one application interface
Strategy	PricingStrategy, RegularPricing, StudentDiscountPricing, GroupDiscountPricing, PriceCalculator	Existing: preserved unchanged and integrated by BookingService

3.2 Factory Method

ReportExporter is the product interface. Each concrete product serializes the same Summary. ExporterFactory is an abstract Creator, while DefaultExporterFactory is its concrete subclass. The public export operation invokes createExporter, so object creation can vary independently of the operation. This explicit creator hierarchy avoids describing a static switch alone as the entire Factory Method pattern. The concrete creator accepts a normalized format string.

```
public interface ReportExporter {
    void export(Summary summary, Path path) throws IOException;
}

public abstract class ExporterFactory {
    public abstract ReportExporter createExporter(String format);

    public final void export(Summary summary, Path path, String format)
            throws IOException {
        createExporter(format).export(summary, path);
    }
}
```

The concrete method returns TxtExporter for txt, JsonExporter for json and HtmlExporter for html; unknown formats fail before touching the destination. ExporterFactoryTest also substitutes a custom creator to prove the export operation dispatches through the overridden method. Adding a new built-in format requires a new product, a concrete-creator mapping and a GUI option; the facade and existing exporters do not change.

3.3 Facade

EventFacade constructs and coordinates the service layer and exposes createEvent, addParticipant, addBooking, changeStatus, getSummary and exportSummary. The GUI imports the facade and data types, without directly calling storage, pricing implementations or exporters.

```
public BookingDetails addBooking(String eventId, String participantId,
                                 int seats, PricingPlan plan) {
    return bookings.addBooking(eventId, participantId, seats, plan);
}

public BookingDetails changeStatus(String bookingId, BookingStatus status) {
    return bookings.changeStatus(bookingId, status);
}

public Summary getSummary(String eventId) {
    return events.getSummary(eventId);
}

public void exportSummary(String eventId, String format, Path path)
        throws IOException {
    exporters.export(getSummary(eventId), path, format);
}
```

These excerpts correspond to the actual classes; only imports, unrelated methods and comments are omitted. Constructor injection of the store and factory supports isolated testing and alternative creators without exposing those details to the GUI.

4. System Architecture

```
Main / SampleData
        |
Swing GUI -> EventFacade
               |-- EventService -> InMemoryEventStore
               |-- BookingService -> PriceCalculator -> PricingStrategy
               |                   -> InMemoryEventStore
               |-- ExporterFactory -> ReportExporter -> output file
```

The existing booking and pricing packages remain intact. New packages are model, storage, service, export, facade, sample and ui. Immutable records carry event, participant and booking details. Summary copies its collections and contains immutable elements, so a later cancellation cannot retroactively modify an already obtained summary.

BookingService validates the event, participant, number of seats, plan and available capacity before saving. A lock around the entire operation prevents two simultaneous requests from both claiming the same remaining seats. Summary creation uses the same store lock. Storage is local to one facade session and is not a database or cross-process transaction system.

5. Implementation Details

5.1 Pricing, participants and status rules

Each participant is a registered booking holder. Group bookings reserve multiple seats without requiring a separate record for every guest. IDs distinguish participants with identical names. Prices use the existing BigDecimal calculations: regular full price, student 10% discount, and group 15% discount when at least five seats are booked. Quoted totals are retained per booking.

CREATED and CONFIRMED both reserve capacity and contribute to the summary total. CANCELLED retains history but releases seats and contributes zero. Reactivation is allowed only when the original seat count fits. Reapplying the same active status does not reserve seats a second time.

5.2 Export and error handling

All reports use UTF-8. JSON escapes quotes, backslashes, control characters and surrogate code units. HTML escapes dynamic text, keeping user-entered markup as text. Monetary values are decimal numbers; the exported currency is KZT. Cancelled bookings remain visible with their quoted price while the report total excludes them.

Export writes to a temporary file beside the target and then replaces the target, using an atomic move where available. Temporary files are cleaned up on failure. The GUI confirms replacement of an existing report and uses a background worker for file output. Validation and I/O exceptions are shown through ErrorDialog, with actionable messages and no stack trace in the user-facing dialog.

5.3 GUI and navigation

The sidebar switches between Events, Participants, Bookings and Summary. A shared event selector keeps the current context visible. Tables are read-only, booking actions refresh totals, and an empty session disables summary export until an event exists.

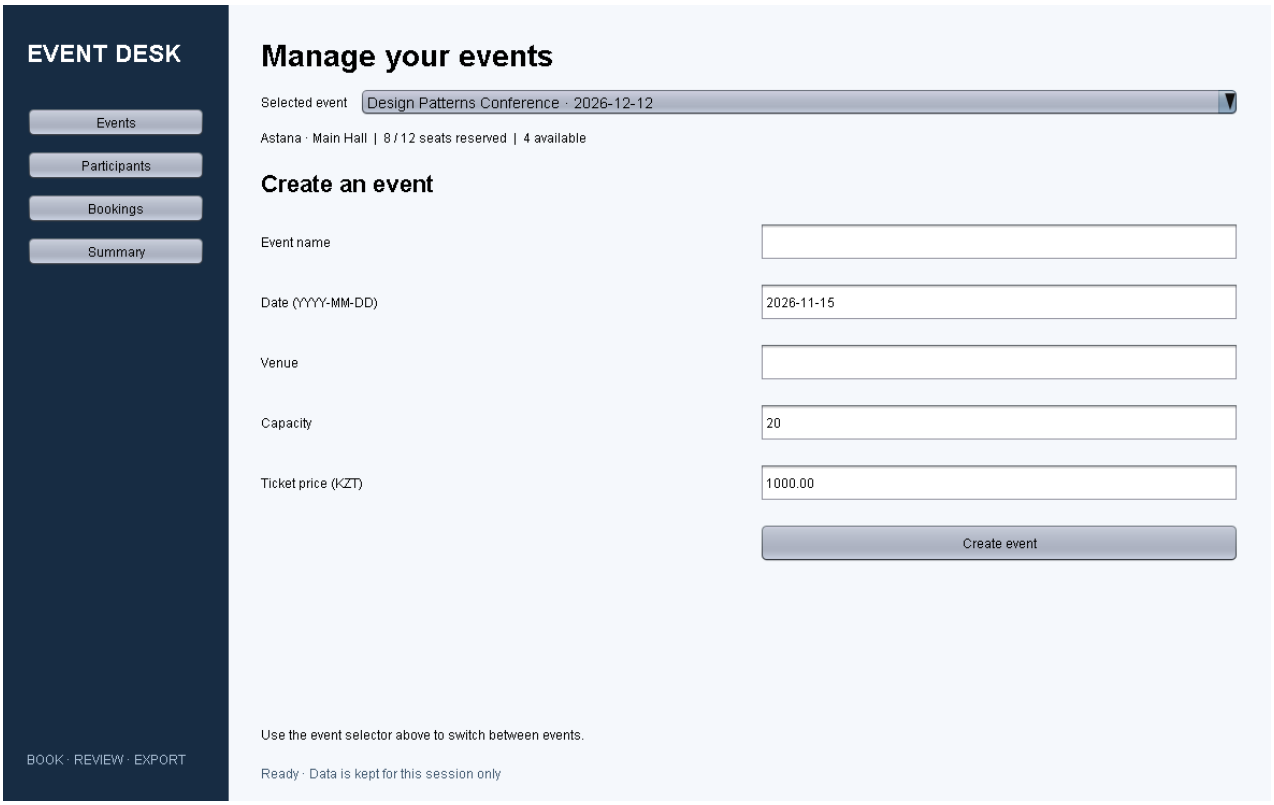


Figure 1. Event creation form and shared navigation. Image captured from actual Swing components.

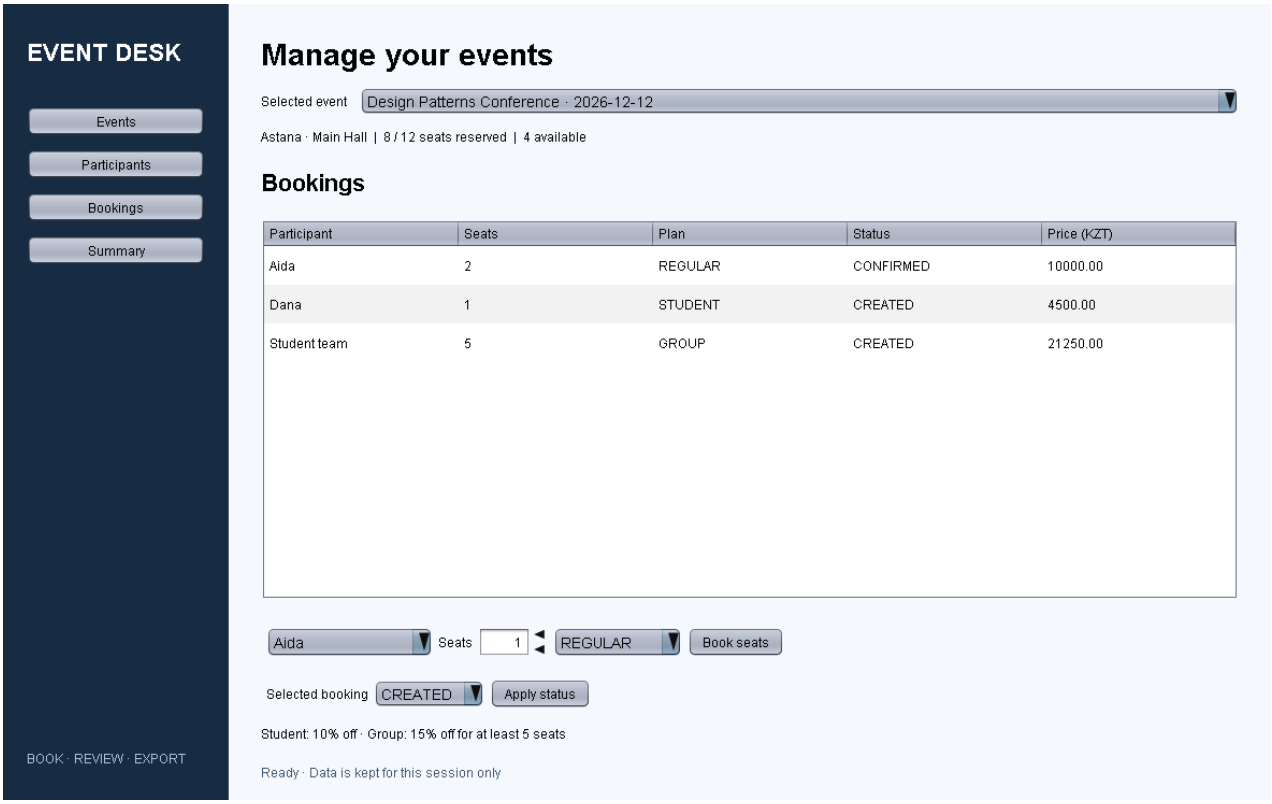


Figure 2. Booking list, participant selection, pricing plan and status actions.

6. User Scenarios

Scenarios 1-6 cover event creation, participant registration, regular/student/group booking and status changes. Full step-by-step instructions are in SCENARIOS.md. The required scenarios 7 and 8 are described below with concrete expected outcomes.

6.1 Scenario 7: export the selected event summary

Launch the demo, select Design Patterns Conference and open Summary. The rows are Aida, two regular seats at 10000.00; Dana, one student seat at 4500.00; and Student team, five group seats at 21250.00. The resulting total is 35750.00 KZT, with eight reserved and four available seats.

Select TXT, JSON or HTML, press Export report and choose a writable destination. Each exported file includes the event, participant identifiers, booking details and the same total. Repeat for all three formats. Cancelling the file chooser should create no file; declining replacement should preserve an existing file. These native-dialog interactions remain a manual checklist.

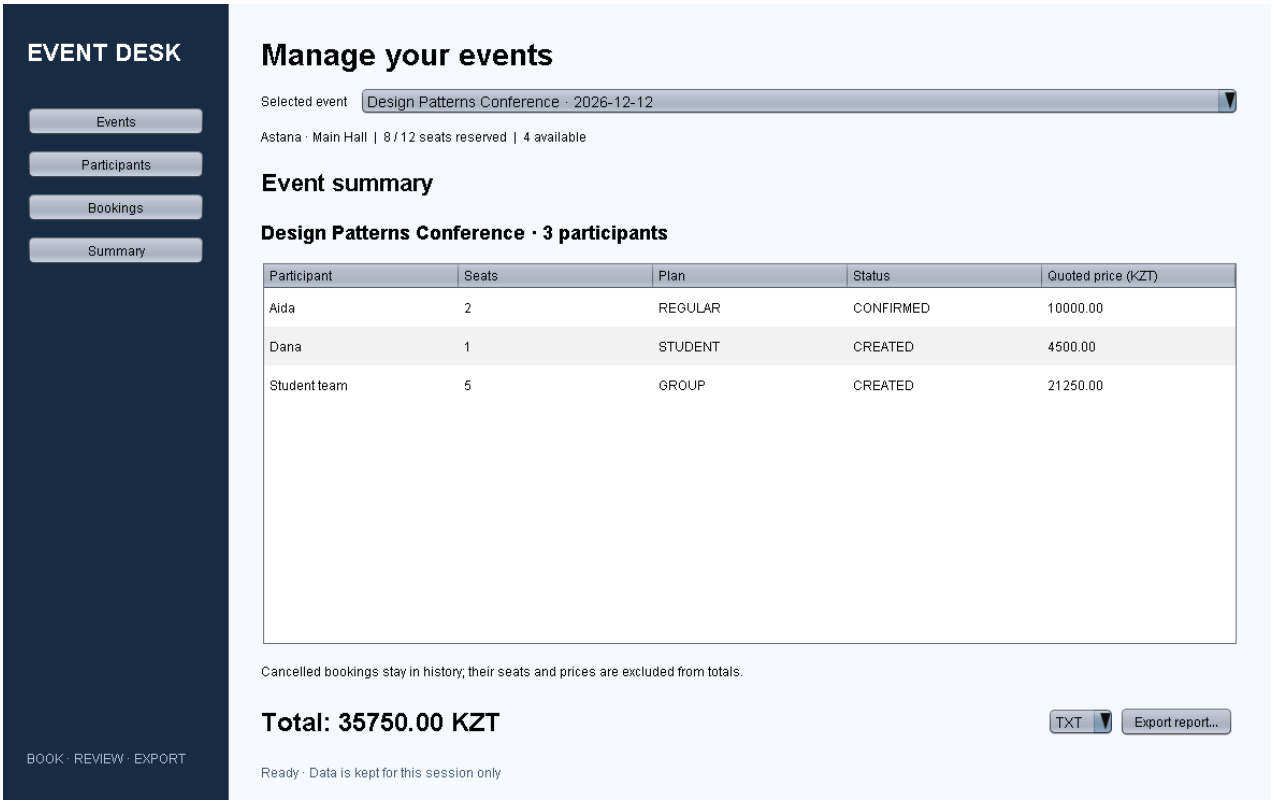


Figure 3. Summary and export controls for the sample conference.

6.2 Scenario 8: booking beyond capacity

Select Java Workshop, which already reserves all three seats. Attempt to book one additional REGULAR seat. The service rejects it with requested 1, available 0. The shared error dialog is displayed; no extra booking is saved and the original 6000.00 KZT total remains unchanged.

Cancel the original booking to release its seats. After adding a new one-seat booking, trying to restore the original three-seat booking fails because only two seats remain. This guards the status-change path as well as the initial booking path.

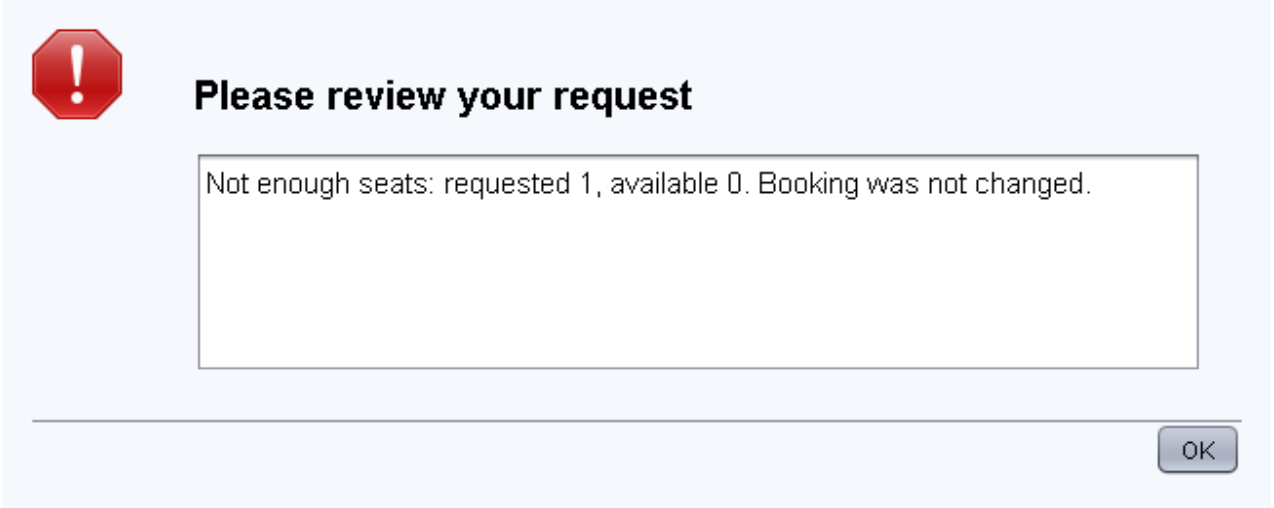


Figure 4. Actual shared error component populated with the exception raised by scenario 8.

7. Testing and Results

The project was compiled and tested locally with OpenJDK 25.0.1 and Maven. The verification suite reports 21 tests, zero failures, zero errors and zero skipped tests.

Test suite	Tests	Coverage
PriceCalculatorTest	2	Existing pricing strategies and minimum group size
ExporterFactoryTest	4	Product types, invalid formats, overridden creator dispatch and escaping
EventFacadeTest	13	Three export formats, overflow, cancellation/reactivation, validation, snapshots, independent events, sample data, I/O failure, overwrite and concurrency
EventPanelTest	2	Navigation, booking/status button actions and empty-session behavior

The build produces a runnable JAR with org.example.Main as its manifest entry point. GitHub Actions is configured to repeat Maven verification on Java 25 when the branch is pushed. No remote CI result is claimed in this report. The original booking/pricing code and its tests were preserved unchanged.

Screenshots were rendered using printAll on actual Swing components on the event dispatch thread. They are reproducible offscreen captures, not mockups. They do not prove desktop window behavior or operating-system file chooser interaction. Those limits and manual checks are explicitly recorded in SCENARIOS.md. No performance benchmark is claimed.

8. Conclusion

This contribution turns the existing booking and pricing foundation into a runnable event management demonstration. Factory Method separates report creation from export use; Facade provides the GUI with a consistent entry point; existing strategies continue to calculate prices. Capacity checks apply both to new bookings and reactivation, while summaries preserve a stable view of the data. Automated tests establish the expected behavior for the required scenarios.

The main limitation is session-only storage. The application is suitable for demonstrating design patterns and local workflows, not yet for a shared production booking service. The report documents these boundaries instead of attributing unrelated functionality from the example.

9. Future Work

- Add durable storage with migrations and transaction-level capacity constraints.

- Add participant contact details, distinct group guest lists and booking search.

- Introduce event lifecycle rules, richer status transitions and configurable pricing policies.

- Add report import, PDF export and a format registry if the set of exporters grows.

- Add end-to-end desktop tests for file selection, cancellation and overwrite confirmation.

- Add localization, accessibility review and keyboard navigation checks on target platforms.

- Add authentication, audit history and multi-user concurrency only when a shared service is needed.

10. Reproduction

```
mvn clean verify
java -jar target/final_project_sdp-1.0-SNAPSHOT.jar
```

Use --empty for an empty session. Use the ScreenshotCapture test utility described in README to regenerate the images and sample TXT/JSON/HTML reports. This report uses the supplied Final Report.pdf only as a reference for numbered sections, pattern explanations, code excerpts, test results, conclusion and future-work structure.